Neural Networks Lecture 2

Data Science Toolbox Week 7

Last time’s Learning Objectives

- Learn what a perceptron is
- Learn what a multilayer perceptron is
- Learn how to measure loss for a neural network
- Learn how gradient descent and back propagation work

Learning Objectives

- Discuss some of the challenges of applying neural networks to computer vision tasks and how they can be addressed with Convolutional Neural Networks.
- Discuss some of the challenges of applying neural networks to language tasks and how they can be addressed with Transformers.

Image specific problems

1. If we represent an image with a flattened vector, a network doesn't know that pixels 10 and 11 are next to each other – how can it learn spatial patterns like edges, corners or textures?
2. If you take an image of a cat and shift every pixel one to the left, the input layer changes massively but the network still has to classify "catness" – how might it do that?
3. A 128x128 RGB image has 49152 values associated with it, how can we practically process images with networks without massively overparameterizing?
4. A local pattern like a vertical edge might appear anywhere on the image, but a standard NN would have to independently learn the same pattern in every location – how might we adjust for this?

Convolutional Neural Networks (CNNs)

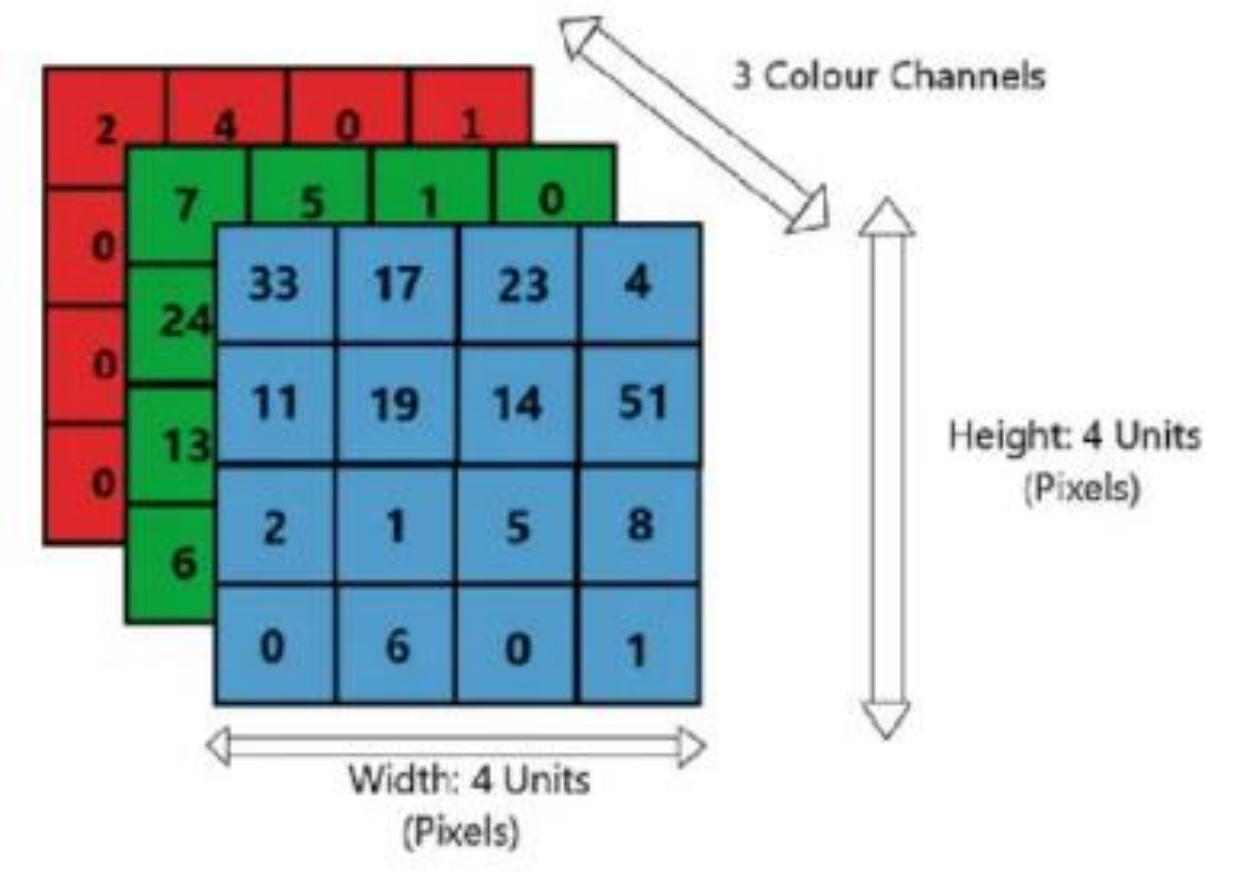
CNNs typically rely on a process of applying:

1. Convolutional Layers
2. Pooling Layers
3. Flattening vectors and putting them through regular linear layers (with non-linear activation functions)

Every layer in a neural network is trying to compress data from higher dimensional space to lower dimensional space.

https://poloclub.github.io/cnn-explainer/

How do we represent an image?



- A colour image is stored as a 3D Tensor of dimensions 3 x height x width.
- The three is for Red, Green, Blue (RGB) values for an image.
- Greyscale images are just 1 x height x width.

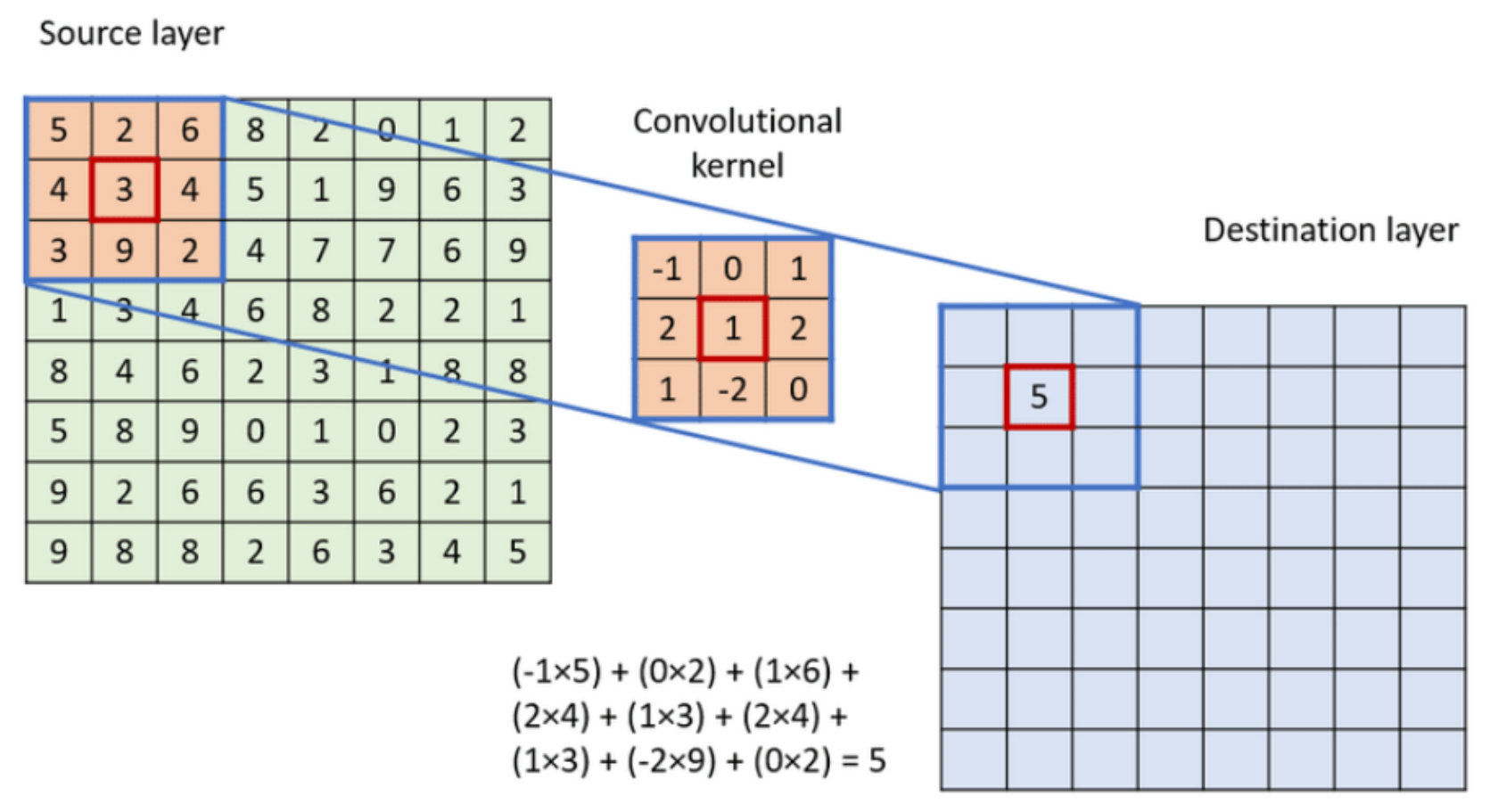
Convolutional Kernel Layers – Part 1

- Pattern extraction layers, looking for abstract patterns like vertical edges or corners in earlier parts of the network as well as more concrete patterns in deeper parts of the network such a face or a wheel depending on context.
- Each kernel is a small matrix that you slide (or “convolve”) over the whole image, right to left, top to bottom. At each position, you perform the dot product for each number the kernel covers and sums the result.
- Each result is one value in the output feature map. The whole feature map tells you where your vertical edge/wheel appears. A convolutional layer might have 1000s of feature maps (in the same way that an RGB image has 3 “feature maps” at input).

Convolutional Kernel Layers – Part 2

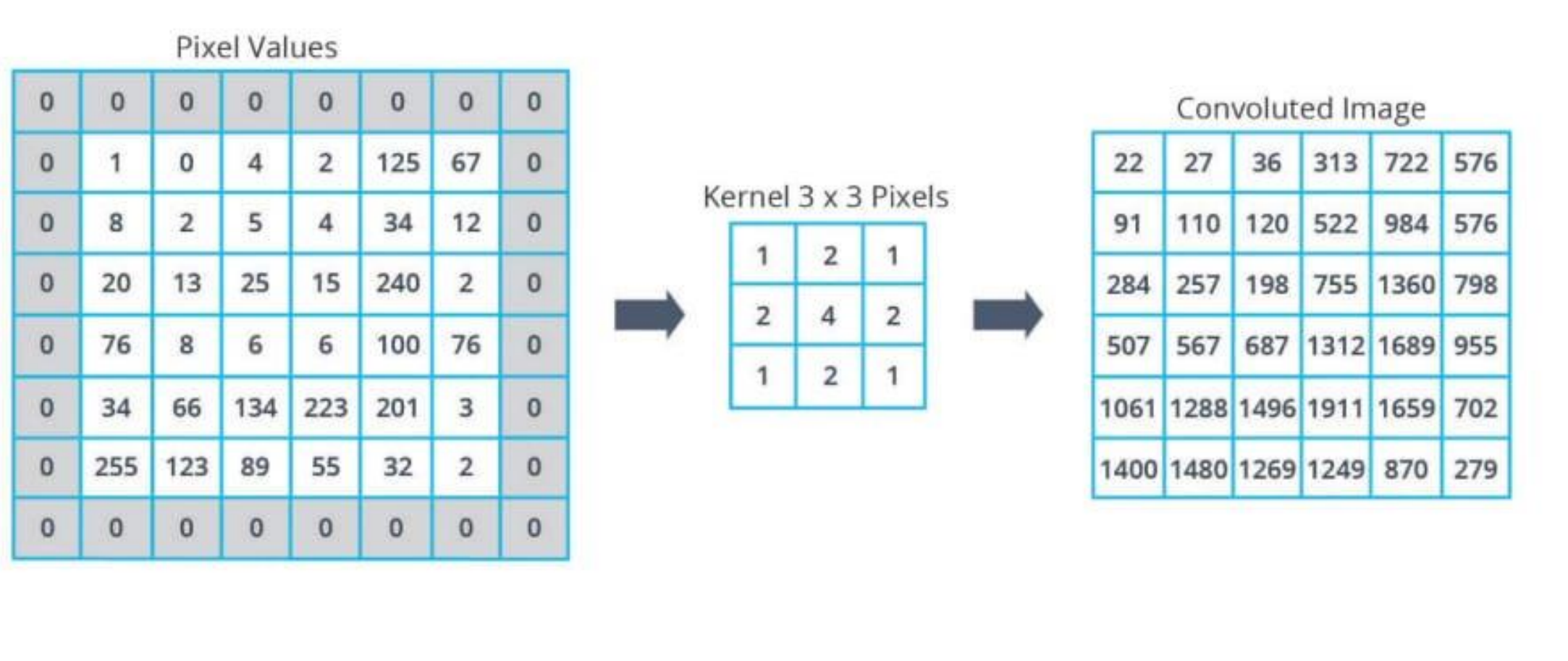
- A single convolutional kernel layer typically learns multiple features at once (e.g. 32 or 64, similar size to your hidden layers) so your output of the convolutional kernel layer will tell you about lots of different patterns.
- The numbers in the kernel layer are learnable parameters, just like the weights and biases of a regular linear layer.
- Kernels have hyperparameters like size, stride and padding.

Kernel Calculation



https://medium.com/@abhishekjainindore24/all-about-convolutions-kernels-features-in-cnn-c656616390a1

Padding for Kernels



https://medium.com/@abhishekjainindore24/all-about-convolutions-kernels-features-in-cnn-c656616390a1

Examples of real Kernels

Original	*Gaussian Blur*	*Sharpen*	*Edge Detection*
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$
			

https://medium.com/@abhishekjainindore24/all-about-convolutions-kernels-features-in-cnn-c656616390a1

Pooling Layers

- In order to further reduce the dimensionality of our data, we can use pooling layers where we attempt to summarise our image based on groups of values.
- Given a kernel size and stride, you can select what sort of pooling operation you want (common ones are max pooling, which returns the maximum across your kernel, or average pooling).

Max Pool Example

1	0	2
3	1	2
1	2	3

Max = 3

1	0	2
3	1	2
1	2	3

Max = 2

1	0	2
3	1	2
1	2	3

Max = 3

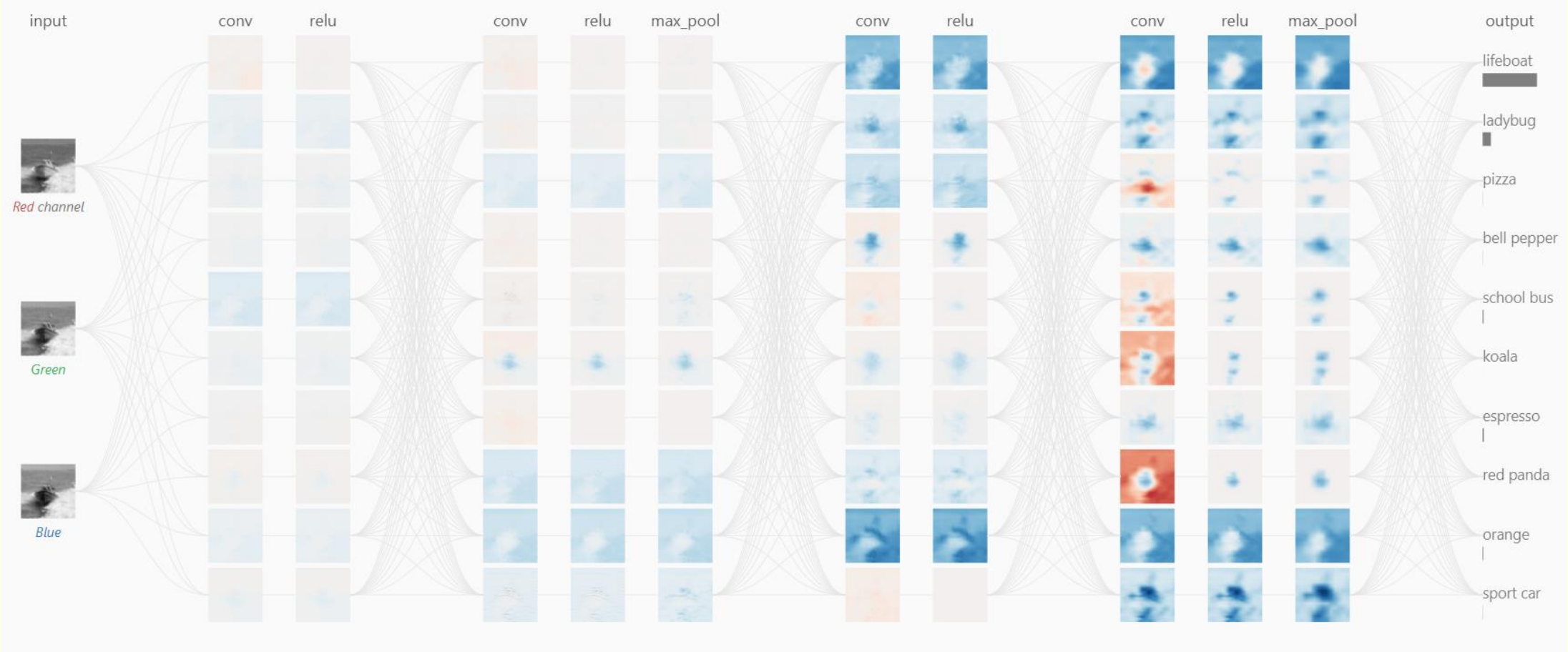
1	0	2
3	1	2
1	2	3

Max = 3

Output:

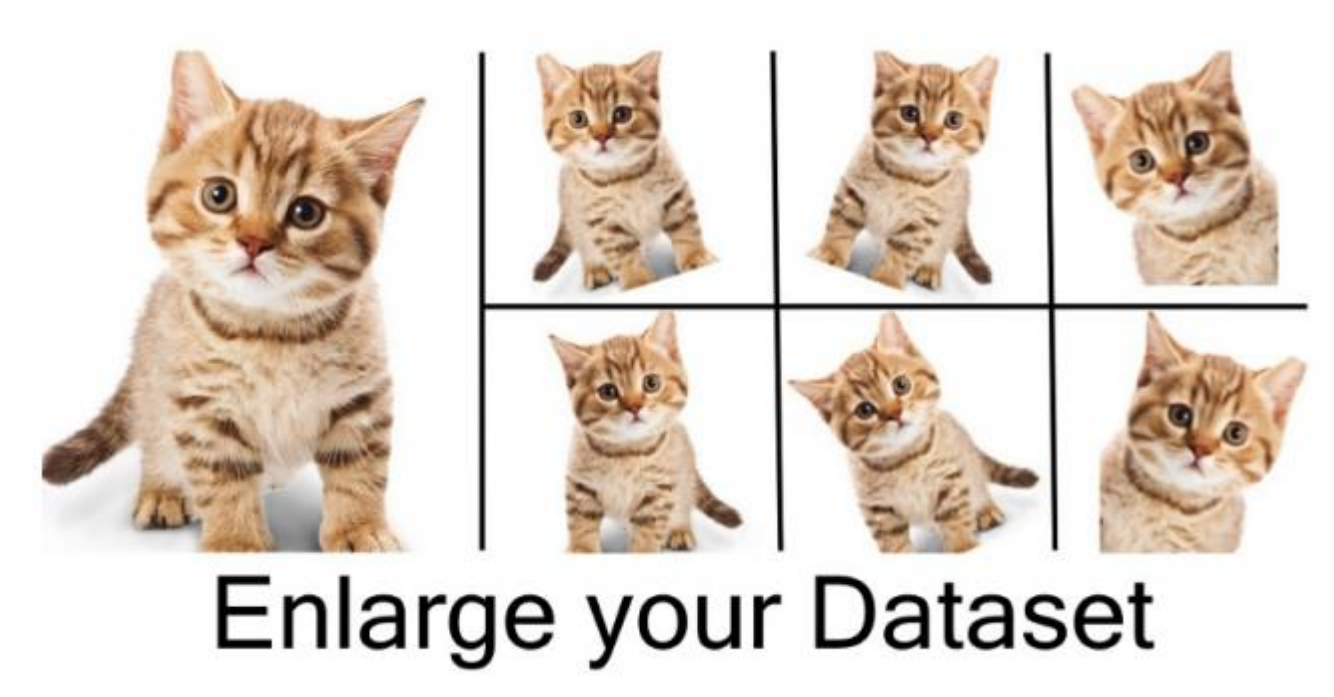
3	2
3	3

The full pipeline:



https://poloclub.github.io/cnn-explainer/

Data Augmentation

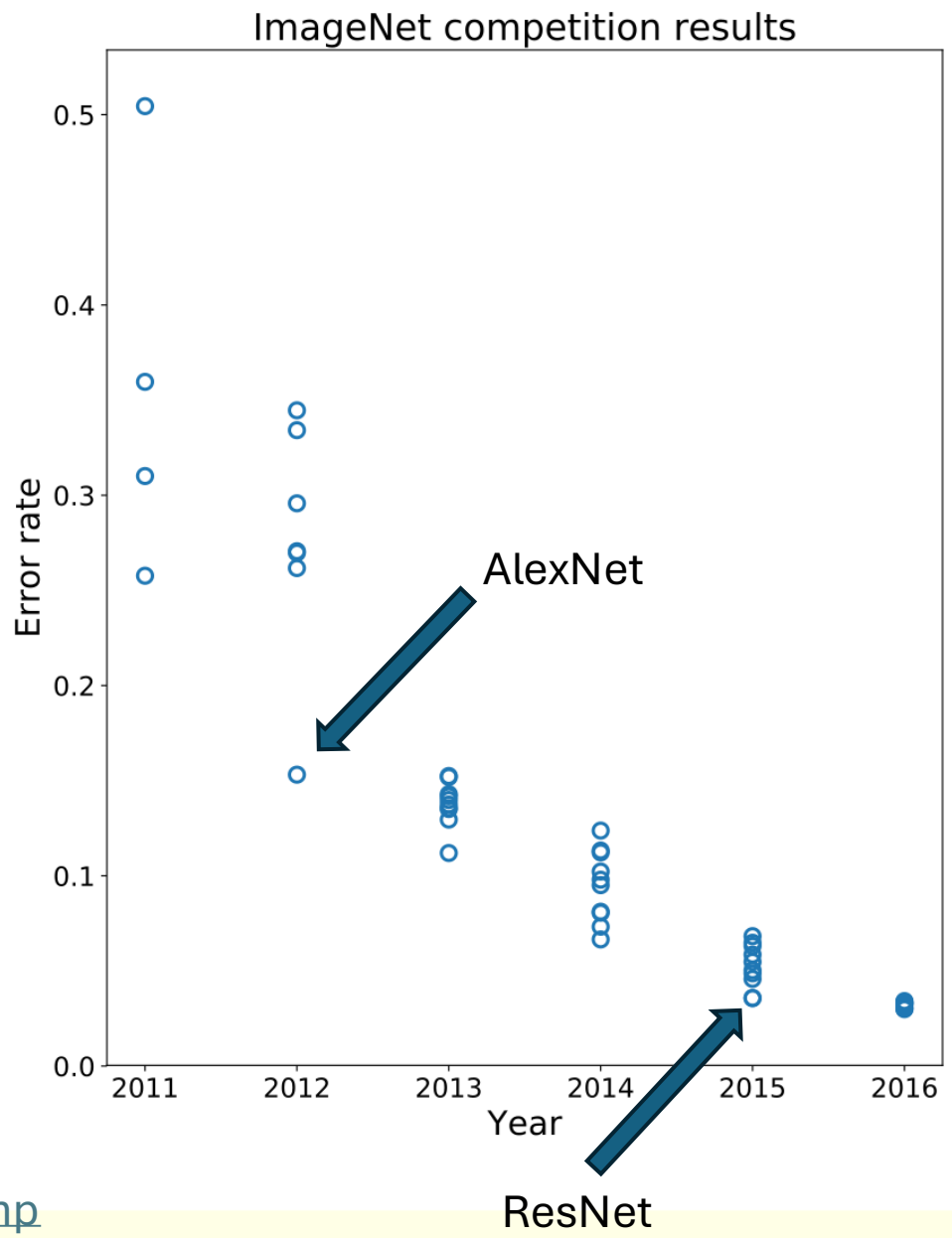


- When using a CNN for image classification, we want our models to be robust to image transformations which represent the same thing.
- If you take an image of a kitten and:
  - Horizontally flip
  - Rotate it
  - Translate/crop it

  It's still a kitten!
- This is a great way to enlarge your training data and help your CNN generalize better.

ImageNet Challenge

- AI researcher Fei-Fei Li assembled a team to collect a large dataset of labelled images for classification.
- Their team scraped the internet for images and employed Mechanical Turks to the label the over 14 million images with more than 20000 categories.
- The ImageNet Project ran a challenge where software programs compete to correctly classify objects.



https://www.image-net.org/challenges/LSVRC/index.php

Discussion: what is attention?